CSA1413

COMPILER DESIGN

Name: Haresh Kumar N L

Reg No: 192425009

Generate Three Address

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100
#define SIZE 100

char opStack[MAX];
char valStack[MAX][SIZE];
int opTop = -1, valTop = -1;
int tempCount = 1;

int precedence(char op)
{
   if (op == '+' || op == '-')
      return 1;
   if (op == '*' || op == '/')
      return 2;
   return 0;
}

void pushOp(char op)
```

```
{
    if (opTop < MAX - 1)
        opStack[++opTop] = op;
}

char popOp(void)
{
    if (opTop >= 0)
        return opStack[opTop--];
    return '\0';
}

char peekOp(void)
{
    if (opTop >= 0)
        return opStack[opTop];
    return '\0';
}

void pushVal(const char *value)
{
    if (valTop < MAX - 1)
    {
        strncpy(valStack[++valTop], value, SIZE - 1);
        valStack[valTop][SIZE - 1] = '\0';
    }
}

void processOperator(void)
```

```
{
    if (opTop < 0 || valTop < 1)
        return;

    char op = popOp();

    char right[SIZE], left[SIZE], temp[SIZE];

    strcpy(right, valStack[valTop--]);
    strcpy(left, valStack[valTop--]);

    sprintf(temp, "t%d", tempCount++);

    printf("%s = %s %c %s\n", temp, left, op, right);

    pushVal(temp);
}

int main(void)
{
    char expr[MAX];
    char target[SIZE] = "";
    int i = 0;

    printf("Enter an arithmetic expression:\n");

    if (fgets(expr, sizeof(expr), stdin) == NULL)
        return 1;
```

```
/* Find the assignment operator */
char *equal = strchr(expr, '=');
char *rhs = expr;

if (equal != NULL)
{
   int len = (int)(equal - expr);
   int j = 0;

   for (int k = 0; k < len && j < SIZE - 1; k++)
   {
      if (!isspace((unsigned char)expr[k]))
         target[j++] = expr[k];
   }

   target[j] = '\0';
   rhs = equal + 1;
}

printf("\nThree Address Code (TAC):\n");

while (rhs[i] != '\0')
{
   if (isspace((unsigned char)rhs[i]))
   {
      i++;
   }
   else if (isalnum((unsigned char)rhs[i]) ||
         rhs[i] == '_')
```

```
{
   char operand[SIZE];
   int j = 0;

   while (isalnum((unsigned char)rhs[i]) ||
         rhs[i] == '_')
   {
      if (j < SIZE - 1)
         operand[j++] = rhs[i];

      i++;
   }

   operand[j] = '\0';
   pushVal(operand);
}
else if (rhs[i] == '(')
{
   pushOp(rhs[i++]);
}
else if (rhs[i] == ')')
{
   while (opTop >= 0 && peekOp() != '(')
      processOperator();

   if (opTop >= 0)
      popOp();

   i++;
```

```
    }
    else if (rhs[i] == '+' || rhs[i] == '-' ||
             rhs[i] == '*' || rhs[i] == '/')
    {
        while (opTop >= 0 && peekOp() != '(' &&
               precedence(peekOp()) >= precedence(rhs[i]))
        {
            processOperator();
        }

        pushOp(rhs[i++]);
    }
    else
    {
        printf("Invalid character: %c\n", rhs[i]);
        return 1;
    }
}

while (opTop >= 0)
{
    if (peekOp() == '(')
    {
        printf("Error: Unmatched parenthesis.\n");
        return 1;
    }

    processOperator();
}
```

```
    if (valTop != 0)
    {
        printf("Error: Invalid arithmetic expression.\n");
        return 1;
    }

    if (target[0] != '\0')
        printf("%s = %s\n", target, valStack[valTop]);

    return 0;
}
```

Output:

